A BEGINNER'S TECHNICAL DEEP-DIVE

Linux Kernel Modules

From Writing Your First Module to Understanding Kernel Source Code

Topics: Building Modules · insmod · lsmod · rmmod · Linked Lists · kernel/module.c

ENVIRONMENT

Kernel: Linux 6.6.14 · Platform: VirtualBox · Architecture: x86_64

`Source file studied: kernel/module.c`

Written for knowledge transfer and public reference. Designed for readers new to kernel development.

Table of Contents

Introduction 2
Chapter 1 — What Is a Kernel Module? 3
1.1 The Two Approaches: Built-in vs Loadable 3
1.2 Where Modules Sit in the System 3
Chapter 2 — Building Your First Module 4
2.1 The Source File (hello.c) 4
2.2 The Makefile 4
2.3 What the Build Produces 5
2.4 What Is Inside a .ko File? 5
Chapter 3 — How insmod Works 6
3.1 The Journey: From Your Terminal to the Kernel 6
3.2 Stage 1: Copy the File into Kernel Memory 6
3.3 Stage 2: Check It Is a Valid ELF File 7
3.4 Stage 3: Find the Module's Identity Card 7
3.5 Stage 4: Version Magic Check 8
3.6 Stage 5: Allocate Memory for the Module 8
3.7 Stage 6: Fix the Addresses (Relocations) 9
3.8 Stage 7: Look Up External Functions 9
3.9 Stage 8: Security Check 10
3.10 Stage 9: Add to the Module List 10
3.11 Stage 10: Call Your init Function 11
3.12 The struct module — the Module's Identity Card 11
3.13 What You See in dmesg 12
Chapter 4 — How Modules Are Stored in the Kernel 13
4.1 The Linked List Idea 13
4.2 The list_head Pattern 13
4.3 The Global modules List 14
4.4 Walking the List: container_of() 14
Chapter 5 — How lsmod Works 15
Chapter 6 — How rmmod Works 16
6.1 The Full Removal Flow 16
6.2 Step 1: Find the Module by Name 16
6.3 Step 2: Check Nobody Is Using It 17
6.4 Step 3: Mark It as Leaving 17
6.5 Step 4: Notify the Rest of the Kernel 17
6.6 Step 5: Call Your exit Function 18
6.7 Step 6: Remove and Free Everything 18
6.8 The Pointer Surgery 19
Chapter 7 — Custom Modules We Built 20
7.1 Module 1: lsmod Replicator 20

7.2 Module 2: Modifying kernel/module.c Directly 21
Chapter 8 — Kernel 3.9 Compatibility Notes 25
Chapter 9 — Lifecycle Summary and Quick Reference 27

Introduction

Have you ever wondered what actually happens when you type sudo insmod hello.ko and press Enter? The module appears in lsmod, your printk message shows up in dmesg, and then rmmod makes it disappear — but what is the kernel doing in between?

This document answers that question from start to finish, using the actual Linux kernel source code. It is written for someone who is new to kernel development but comfortable reading C. Every concept is explained in plain English first, then illustrated with a short code snippet showing exactly which function in the kernel source makes it happen.

We built and tested everything described here on Linux kernel 3.9.0 inside VirtualBox. Where code differs between kernel 3.9 and modern kernels, both versions are shown.

What This Document Covers

Chapter	Topic
1	What a kernel module is and how it fits into Linux
2	Writing, compiling, and inspecting a kernel module
3	How insmod works — all ten stages, with the source function at each step
4	How loaded modules are stored in a linked list inside the kernel
5	How lsmod reads and displays the list of loaded modules
6	How rmmod safely removes a module — step by step through the source
7	Two custom modules we built: an lsmod replicator and a live kernel trace
8	Compatibility notes for kernel 3.9 vs. modern kernels
9	Complete lifecycle diagram and command quick reference

How to Read This Document

Each section follows the same pattern: a plain-English explanation of what is happening, sometimes an analogy, and then a short code excerpt showing where in kernel/module.c it happens. You do not need to understand every line of C — the key thing is to know which function does what, so you can find it and read it yourself.

Box Type	Meaning
Info box (blue border)	Extra background or explanation — useful but not critical
Warning box (red border)	Something important that causes failures if ignored
Tip box (green border)	A design insight or a helpful way to remember something

CHAPTER 1

What Is a Kernel Module?

The Linux kernel is the core of the operating system. It manages your CPU, RAM, disk, and every other piece of hardware. When a new device comes along — say, a USB Wi-Fi adapter — the kernel needs code to talk to it. There are two ways to add that code.

1.1 The Two Approaches: Built-in vs Loadable

Approach	What it means	Problem
Built into the kernel	Compile the driver code directly into the kernel image that boots	Every change requires recompiling the whole kernel and rebooting the machine
Loadable kernel module (.ko)	Compile it as a separate file that can be plugged into the running kernel at any time	None — this is why modules exist

A kernel module is a compiled file with the extension .ko (Kernel Object). You load it with insmod, the kernel runs it, and you remove it with rmmod — all without rebooting. Device drivers, file systems, and network protocols are all commonly shipped as modules.

> **ℹ SIMPLE ANALOGY**
> Think of the Linux kernel as a smartphone. The apps you install and remove are like kernel modules — they extend what the phone can do without replacing the operating system. The kernel is the OS; a module is an extension of the OS itself.

1.2 Where Modules Sit in the System

This is the most important thing to understand before writing a module: modules do not run as normal programs. They run inside the kernel, with the same privileges as the kernel itself. There is no safety net.

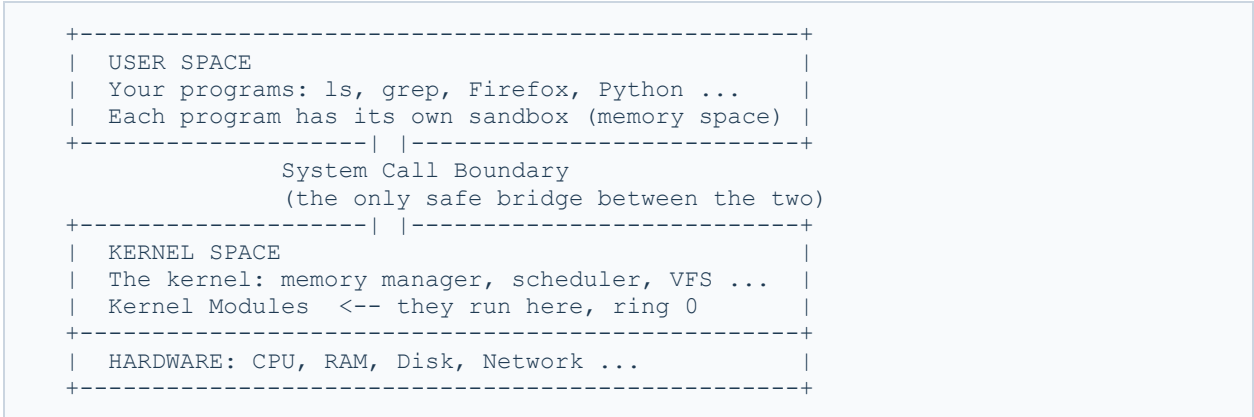


Figure: System layers — modules execute in kernel space, not user space

> **⚠ KERNEL SPACE HAS NO SAFETY NET**
> A bug in a user-space program causes that program to crash, but nothing else. A bug in a kernel module — a bad pointer, an infinite loop — crashes the entire machine immediately. This is why you always develop and test kernel modules inside a virtual machine first.

CHAPTER 2

Building Your First Module

Let's write, compile, and inspect a real kernel module. This chapter covers all three steps.

2.1 The Source File (hello.c)

Every kernel module needs at minimum two functions: one that runs when the module is loaded (init), and one that runs when it is removed (exit). You register them with the kernel using the module_init() and module_exit() macros.

```c
/* hello.c */
#include <linux/module.h>   /* required for every module             */
#include <linux/kernel.h>   /* for printk() and the KERN_* log levels */
#include <linux/init.h>     /* for the __init and __exit annotations  */

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("My first kernel module");

static int __init hello_init(void)
{
    /* printk() is the kernel's printf — output goes to dmesg */
    printk(KERN_INFO "[hello] Module loaded.\n");
    return 0;    /* returning 0 means success; non-zero = load failed */
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "[hello] Module removed.\n");
}

/* Tell the kernel which functions are our init and exit */
module_init(hello_init);
module_exit(hello_exit);
```

WHAT IS PRINTK?

printk() works like printf() but writes into the kernel's ring buffer — a circular in-memory log. KERN_INFO is the priority level (like a log level). You read these messages from the terminal with the dmesg command.

WHAT DOES __INIT DO?

The __init annotation tells the compiler to place that function in a special memory section (.init.text). Once the module has finished loading, the kernel frees that section — it is never needed again. This saves memory. The same idea applies to __exit.

2.2 The Makefile

You do not compile a kernel module by calling gcc directly. You use the kernel's own build system (called kbuild), which knows all the right compiler flags and settings for your kernel version.

```
# Makefile
obj-m += hello.o
```

```
all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Line	What it means
obj-m += hello.o	Tell kbuild: compile hello.c into a loadable module (not built-in)
-C /lib/modules/.../build	Jump into the kernel's own build directory and use its rules
M=$(PWD)	But put the output files back here, in our current folder
modules	The build target that produces a .ko output file

2.3 What the Build Produces

After running make, you will find several new files in your directory. Only hello.ko matters for loading; the others are intermediate build artefacts.

File	Purpose
hello.ko	The kernel module — this is what you pass to insmod
hello.o	Compiled object file, an intermediate step
hello.mod.c	Auto-generated glue code that adds metadata (author, vermagic, etc.)
Module.symvers	Exported symbol list — used if another module depends on this one

2.4 What Is Inside a .ko File?

A .ko file is an ELF file — a structured binary container, like a ZIP file, where each section has a name and holds a specific type of content. Here are the sections that actually matter for understanding what insmod does:

```
hello.ko  (think of it like a ZIP file with named pockets)

Section          What is stored there
---------------  ------------------------------------------------
.text            Your actual compiled code (hello_init, hello_exit)
.modinfo         The module's name, license, author, and vermagic
                 (vermagic is the version string the kernel checks)
this_module      A pre-filled identity card (struct module) that
                 the kernel reads to know about this module
.rela.text       A list of addresses that need to be filled in
                 at load time (because addresses are not known yet)
```

Figure: The important sections inside hello.ko

ℹ WHY CAN'T YOU JUST RUN A .KO FILE LIKE A NORMAL PROGRAM?

A .ko has no fixed address and no entry point. It is designed to be inserted into the kernel's memory at whatever address the kernel picks. Running it directly would fail instantly — there is no OS beneath it to support it.

CHAPTER 3

How insmod Works

When you run sudo insmod hello.ko, the insmod tool (a regular userspace program) opens the file, reads it into memory, and then hands it to the kernel via a system call. The kernel then takes over completely. All the real work happens inside a single source file: kernel/module.c.

This chapter walks through every stage, showing the actual function name and a short excerpt from kernel/module.c at each step. You do not need to understand every line — the goal is to know which function does what so you can find it yourself.

3.1 The Journey: From Your Terminal to the Kernel

```
$ sudo insmod hello.ko
        |
        |  insmod (a regular program in /sbin/):
        |    1. Opens hello.ko
        |    2. Reads the file size
        |    3. Maps the file into memory
        |    4. Calls init_module(memory, size, "")  <-- system call
        |
        +------- crosses into kernel space -----------+
                                                      |
        kernel/module.c takes over:                   |
          sys_init_module()   <- syscall entry point  |
            load_module()     <- runs all 10 stages   |
              do_init_module()<- calls hello_init()   |
                                                      |
        Module is live. lsmod will now show it.       |
```

Figure: Full path from terminal command to running module

Let's trace each stage inside load_module().

3.2 Stage 1: Copy the File into Kernel Memory

The kernel cannot safely touch userspace memory directly. The very first thing it does is copy the entire .ko file into its own memory using a function called copy_module_from_user(). Only after this copy is the kernel working with memory it fully controls.

Where to find it: `kernel/module.c` — function `copy_module_from_user()`

```c
/* kernel/module.c — Stage 1: copy_module_from_user() */
/* Stage 1 does two things: */

/* 1. Allocate memory inside the kernel to hold the .ko file */
info->hdr = vmalloc(info->len);

/* 2. Safely copy the .ko bytes from userspace into that memory */
copy_from_user(info->hdr, umod, info->len);
```

WHY COPY AT ALL?

Your program (insmod) lives in user space. The kernel lives in kernel space. These are completely separate memory worlds — a pointer from one side is meaningless on the other. copy_from_user() is the safe, official bridge that moves bytes across that boundary.

3.3 Stage 2: Check It Is a Valid ELF File

Before doing anything else with the file, the kernel checks that what it received is actually a valid module file. Every ELF file starts with the same 4 magic bytes: 0x7f followed by the letters E, L, F. If those bytes are wrong, the file is not a valid module and the load stops immediately.

Where to find it: `kernel/module.c` — function `elf_header_check()`

```
/* kernel/module.c — Stage 2: elf_header_check() */
/* Check 1: Does the file start with the ELF magic bytes? */
/* Every valid ELF file starts with:  0x7f  E  L  F      */
if (first 4 bytes are not 0x7f E L F)
    return -ENOEXEC;    /* not a valid file — stop */

/* Check 2: Is it a relocatable object (.ko type)? */
/* Not a standalone program — it must be loadable   */
if (file_type != ET_REL)
    return -ENOEXEC;
```

3.4 Stage 3: Find the Module's Identity Card

Remember how a .ko file is like a ZIP with named sections? Now the kernel opens that ZIP and looks through the sections one by one, searching for the one that holds the module's identity card. That section is called this_module, and it contains a pre-filled struct module.

Where to find it: `kernel/module.c` — function `setup_load_info()`

```
/* kernel/module.c — Stage 3: setup_load_info() */
/* Look through every section in the .ko file */
for each section in the file:
    if section name is ".gnu.linkonce.this_module":
        info->mod = this section    /* found the identity card */

/* If it was never found, this is not a real kernel module */
if (!info->mod)
    return -ENOEXEC;
```

ℹ WHAT IS A STRUCT MODULE?

struct module is the kernel's identity card for a loaded module. It holds the name, size, pointers to the init and exit functions, and the list node that connects it to all other modules. The compiler created this struct and embedded it in the .ko file when you ran make. Section 3.12 shows what it looks like.

3.5 Stage 4: Version Magic Check

Every .ko file contains a string called vermagic — something like "3.9.0 SMP mod_unload". The kernel compares this against its own version string. If they do not match exactly, the load is rejected immediately.

Where to find it: `kernel/module.c` — function `check_modinfo()`

```
/* kernel/module.c — Stage 4: check_modinfo() */
/* Read the vermagic string from the .modinfo section */
modmagic = get_modinfo(info, "vermagic");

/* Compare it against the running kernel's own version string */
if (strcmp(modmagic, VERMAGIC_STRING) != 0) {
    printk(KERN_ERR "%s: version magic '%s' should be '%s'\n",
           mod->name, modmagic, VERMAGIC_STRING);
```

```
    return -ENOEXEC;    /* hard reject */
}
```

> ⚠ **WHY VERMAGIC IS STRICT**
>
> If you compile a module on kernel 3.9.0 and then try to load it on kernel 3.9.1, even a tiny internal struct layout change in the kernel could make your module read the wrong memory — causing a crash. The vermagic check is what prevents that. Always compile your module on the same kernel you intend to load it on.

3.6 Stage 5: Allocate Memory for the Module

Now the kernel works out how much memory the module needs and allocates it. It splits this into two regions: the core region (permanent — holds the code and data) and the init region (temporary — holds the __init functions, freed after they run).

Where to find it: `kernel/module.c` — function `layout_and_allocate()`

```
/* kernel/module.c — Stage 5: layout_and_allocate() (simplified) */
/* Calculate sizes, then allocate executable kernel memory */
mod->module_core = module_alloc(mod->core_size);  /* permanent region */
mod->module_init = module_alloc(mod->init_size);  /* temporary region */

/* Copy all the sections into the right places */
move_module(mod, info);
```

> ✦ **WHY TWO MEMORY REGIONS?**
>
> Your hello_init() function is marked __init, which places it in the init region. After the module finishes loading, that region is freed — the code no longer exists in memory. This is intentional: init functions run once and are never needed again. The permanent core region holds everything the module needs while it is running.

3.7 Stage 6: Fix the Addresses (Relocations)

When your module was compiled, it did not know where in memory it would be loaded. So the compiler left placeholder values (zeros) wherever an address is needed — for example, at every call to printk(). The relocation step patches each placeholder with the real runtime address.

Where to find it: `kernel/module.c` — function `apply_relocations()`

```
/* kernel/module.c — Stage 6: apply_relocations() — concept */
/* For each item in the relocation list: */
for each item:
    real_address = look_up(function_name_like_printk);
    write real_address into the placeholder slot in the code;
```

> ℹ **RELOCATION IN PLAIN ENGLISH**
>
> When your module calls printk(), the compiler writes 0x00000000 at that call site and notes: "fill in the address of printk here". apply_relocations() looks up printk's real address in the running kernel and writes it into that slot. Without this step, every function call in your module would jump to address zero — an instant crash.

3.8 Stage 7: Look Up External Functions

Your module uses functions that live in the kernel — like printk() and kmalloc(). At compile time, the module does not know the real memory addresses of those functions. This stage looks each one up in the kernel's symbol table and records the address so the module can call them correctly.

Where to find it: `kernel/module.c` — function `simplify_symbols()`

```
/* kernel/module.c — Stage 7: simplify_symbols() */
/* For each function the module uses but does not define itself: */
for each external symbol (e.g. printk, kmalloc):
    address = search_kernel_symbol_table(symbol_name);
    if (address found):
        record address so the module can call it
    else:
        printk("Unknown symbol %s", name);
        return error;   /* unknown function — reject the module */
```

ℹ WHAT IF A SYMBOL IS NOT FOUND?

If your module calls a kernel function that the kernel has not made available to modules, this stage fails with "Unknown symbol" and loading stops. This is the error you see when a module was compiled for a different kernel version or uses a non-exported internal function.

3.9 Stage 8: Security Check

Before the module can run, the Linux Security Module (LSM) framework gets a chance to approve or reject it. If SELinux or AppArmor is active, they check the module against their policies. If module signing is configured, the kernel also verifies a cryptographic signature at the end of the .ko file.

Where to find it: `kernel/module.c` — functions `security_module_check()` and `module_sig_check()`

```
/* kernel/module.c — Stage 8 */
err = security_module_check(mod, info);  /* SELinux / AppArmor */
if (err) goto free_module;

err = module_sig_check(info);            /* signature (if enabled) */
if (err) goto free_module;
```

3.10 Stage 9: Add to the Module List

The module has passed all checks. It is now added to the kernel's global list of loaded modules. The moment this happens, the module becomes visible to lsmod.

Where to find it: `kernel/module.c` — function `list_add_rcu()`

```
/* kernel/module.c — Stage 9: add to global modules list */
mutex_lock(&module_mutex);         /* prevent two insmod at the same time */

if (find_module(mod->name))        /* reject if already loaded */
    { err = -EEXIST; goto unlock; }

mod->state = MODULE_STATE_COMING;  /* visible but init not yet called */
list_add_rcu(&mod->list, &modules);/* insert into the global list */

mutex_unlock(&module_mutex);
```

ℹ WHAT IS RCU?

list_add_rcu() uses a mechanism called Read-Copy-Update (RCU). It allows other parts of the kernel to read the module list at the same time without taking a lock. This makes lsmod fast. Writes (insmod, rmmod) use special RCU-safe functions and wait for all active readers to finish before touching memory.

3.11 Stage 10: Call Your init Function

The last stage is in its own function: do_init_module(). It calls your hello_init() function directly and waits for it to return. If it returns 0, the module state is set to LIVE and loading is complete. If it returns an error, everything is undone.

Where to find it: `kernel/module.c` — function `do_init_module()`

```
/* kernel/module.c — Stage 10: do_init_module() */
static noinline int do_init_module(struct module *mod)
{
    /* do_one_initcall() calls your hello_init() and waits */
    ret = do_one_initcall(mod->init);
    if (ret < 0)
        goto fail_free_freeinit;   /* undo everything */

    mod->state = MODULE_STATE_LIVE;/* module is now fully active */
    free_module_init(mod);         /* free the __init memory section */
    return 0;
}
```

3.12 The struct module — The Module's Identity Card

For as long as a module is loaded, the kernel keeps a struct module in memory to describe it. This is the identity card referenced throughout this document. Each field connects directly to something you already know:

```
/* This is what the kernel holds in memory for every loaded module */
/* include/linux/module.h -- simplified for clarity */
struct module {

    /* Is the module loading, fully live, or being removed? */
    enum module_state  state;

    /* The link that connects it to all other loaded modules */
    struct list_head   list;

    char  name[];          /* "hello" -- the module name             */
    void *module_core;     /* where the code lives in kernel memory  */
    unsigned int core_size;/* how many bytes of memory it uses       */

    int  (*init)(void);    /* points straight to YOUR hello_init()   */
    void (*exit)(void);    /* points straight to YOUR hello_exit()   */
};
```

3.13 What You See in dmesg After insmod

After a successful insmod, running dmesg shows the kernel's log. Two kernel-generated lines appear first, then your own printk output from hello_init().

```
/* dmesg output after insmod */
$ sudo insmod hello.ko
$ dmesg | tail -5

[12345.678] hello: module license 'GPL' taints kernel.
[12345.680] hello: loading out-of-tree module taints kernel.
[12345.681] [hello] Module loaded.
            ^^^^^^^^^^^^^^^^^^^^^^^^
            Your printk() from hello_init()
```

```
/* The first two lines come from Stage 4 (check_modinfo).
   They are warnings, not errors. They appear because this
   module was not part of the upstream kernel source tree. */
```

CHAPTER 4

How Modules Are Stored in the Kernel

Once a module is loaded, the kernel needs to keep track of it so that lsmod can list it, rmmod can find it, and the symbol resolver can search it. The kernel stores every loaded module in a circular doubly-linked list, with all the modules connected to each other.

4.1 The Linked List Idea

ℹ TRAIN ANALOGY

Imagine a train. Each carriage knows the carriage in front of it and the carriage behind it. You can add a carriage anywhere or remove one without rebuilding the whole train — you just update the connections on the two neighbouring carriages. A doubly-linked list works exactly the same way, but with pointers instead of couplings. And because it is circular, the last carriage connects back to the first.

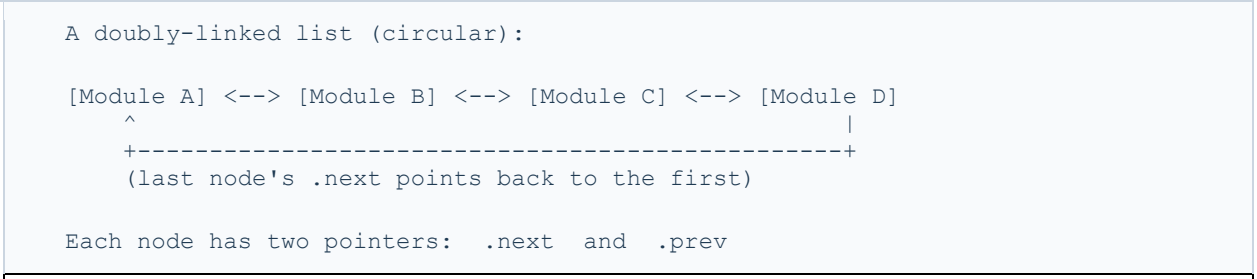


Figure: Circular doubly-linked list — how modules are chained

4.2 The list_head Pattern

Instead of creating a separate list node type, the Linux kernel uses a clever trick: it embeds a tiny two-pointer struct called list_head directly inside whatever struct it wants to connect. For modules, the list_head is embedded inside struct module.

```c
/* The list_head pattern — embedding the list node inside the data */
/* include/linux/list.h */
struct list_head {
    struct list_head *next;
    struct list_head *prev;
};

/* struct module contains one of these: */
struct module {
    struct list_head  list;   /* <-- this is the list node */
    char              name[]; /* the module's name         */
    /* ... */
};
```

✦ WHY EMBED THE NODE INSIDE THE STRUCT?

This design means the exact same list_add / list_del / list_for_each code works for any struct in the kernel — not just modules. The kernel uses this pattern for processes, network devices, file systems, and hundreds more types. It is one of the cleanest patterns in the entire kernel.

4.3 The Global modules List

The kernel declares one global list_head anchor named modules. Every struct module's embedded list node is connected into this circular list when the module loads, and disconnected when it is removed.

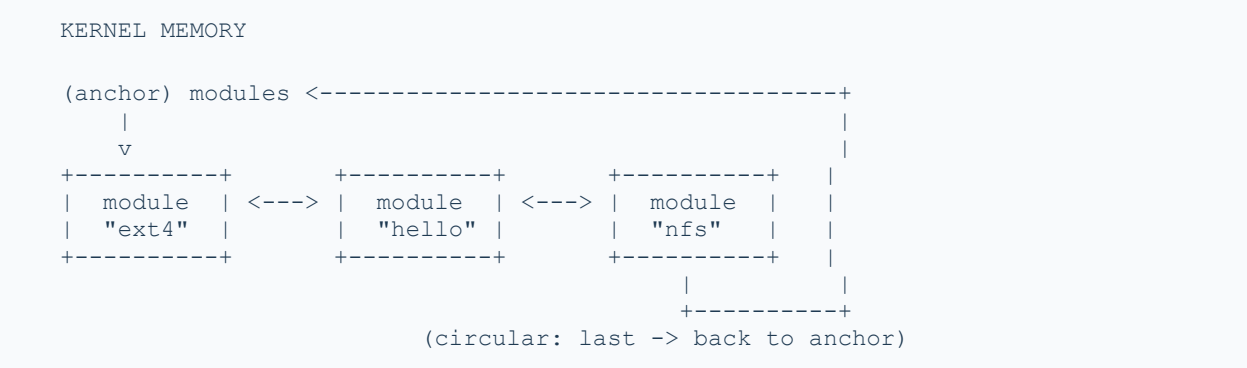


Figure: The global modules circular list in kernel memory

4.4 Walking the List: container_of()

When you traverse the list, each step gives you a pointer to the embedded list_head node — a tiny two-pointer struct. But what you actually want is the whole struct module. The kernel has a macro for this called container_of(), which list_entry() wraps.

Think of it like this: you find a name tag (the list_head) clipped onto a jacket (the struct module). You know exactly which pocket the name tag is clipped to. So to find the jacket, you just step back by that pocket distance. That is all container_of() does.

```
/* list_for_each_entry() -- how lsmod and rmmod walk all loaded modules */
/* Given a list_head pointer, recover the full struct module */
struct module *mod = list_entry(ptr, struct module, list);

/* To walk every module in the global list: */
struct module *mod;
list_for_each_entry(mod, &modules, list) {
    printk(KERN_INFO "  %s ", mod->name); /* print each module name */
}
```

CHAPTER 5

How lsmod Works

The lsmod command is a small userspace tool. All it does is open a file called /proc/modules, read its contents, and format them into the table you see in the terminal. The interesting part is how /proc/modules gets its content.

```
$ lsmod
    |
    |  open("/proc/modules")  — a virtual file, no disk backing
    v
Kernel generates the content on the fly:
    For each module in the global modules linked list:
        print: name  size  refcount  dependencies
    |
    v
Module            Size    Used by
hello             12288   0
ext4              561152  2
...
```

Figure: lsmod reads /proc/modules, which the kernel generates from the modules list

/proc/modules is a virtual file — it does not exist on disk. Every time anything reads it, the kernel walks the global modules linked list and generates the content fresh. The kernel implements this using the seq_file interface, which handles chunked reading correctly for large lists.

Column in lsmod output	Where it comes from in struct module	What it means
Module	mod->name	The module's name
Size	mod->core_size (kernel 3.9) or mod->core_layout.size (kernel 4.5+)	Bytes of kernel memory the module uses
Used by	module_refcount(mod)	How many other modules or users depend on this one

REFCOUNT AND DEPENDENCIES

The "Used by" column shows the reference count. If module B calls a function exported by module A, then A's refcount is at least 1. This is why you cannot rmmod A while B is loaded — and why lsmod shows A's "Used by" as "B".

CHAPTER 6

How rmmod Works

Running sudo rmmod hello triggers the delete_module system call. The kernel must check that the module is safe to remove, call its exit function so it can clean up, unlink it from the global list, and then free all its memory. Everything must happen in the correct order to avoid crashes.

6.1 The Full Removal Flow

```
$ sudo rmmod hello
        |
        |  delete_module("hello", 0)  <- system call
        v
kernel/module.c: sys_delete_module()
  |
  +-- Step 1: find_module("hello")        search the linked list
  +-- Step 2: refcount == 0?              is anyone using it?
  +-- Step 3: state = MODULE_STATE_GOING  mark it as leaving
  +-- Step 4: notifier chain              warn the rest of the kernel
  +-- Step 5: mod->exit()                 call your hello_exit()
  +-- Step 6: free_module(mod)
        |-- list_del_rcu()              unlink from the list
        |-- synchronize_rcu()           wait for all list readers
        |-- module_free(module_core)    free the code from memory
        +-- kfree(mod)                  free the struct module itself
```

Figure: rmmod: complete removal sequence

6.2 Step 1: Find the Module by Name

find_module() does a simple walk of the global modules linked list, comparing each module's name field to the string you passed to rmmod.

Where to find it: `kernel/module.c` — function `find_module()`

```c
/* kernel/module.c — find_module() */
struct module *find_module(const char *name)
{
    struct module *mod;
    list_for_each_entry(mod, &modules, list) {
        if (strcmp(mod->name, name) == 0)
            return mod;    /* found it */
    }
    return NULL;           /* not loaded */
}
```

6.3 Step 2: Check Nobody Is Using It

Before anything else can happen, the kernel checks that the module's reference count is zero. If any other module imports a function from this one, or a process has an open connection to a device it manages, the count is non-zero and rmmod is rejected.

Where to find it: `kernel/module.c` — function `try_stop_module()`

```
/* kernel/module.c — refcount check inside try_stop_module() */
if (module_refcount(mod) != 0) {
    /* someone is still using it — refuse to unload */
    return -EWOULDBLOCK;  /* user sees: "Module is in use" */
}
```

> ⚠ **REMOVING A BUSY MODULE**
> If you try to rmmod a module that something else depends on, you will see: "rmmod: ERROR: Module hello is in use." You must first rmmod every module that depends on it, in reverse dependency order.

6.4 Step 3: Mark It as Leaving

Once the refcount check passes, the kernel changes the module's state to MODULE_STATE_GOING. This is a signal to all other kernel code: stop trying to use this module. After this point, find_symbol() will skip the module's exports and no new references can be taken.

```
/* kernel/module.c — state transition */
mod->state = MODULE_STATE_GOING;

/* The four possible states of a module: */
/* MODULE_STATE_LIVE     : fully loaded, running normally */
/* MODULE_STATE_COMING   : insmod in progress             */
/* MODULE_STATE_GOING    : rmmod in progress              */
/* MODULE_STATE_UNFORMED : allocated, not yet initialised */
```

6.5 Step 4: Notify the Rest of the Kernel

Other kernel subsystems — like ftrace (function tracing) and kprobes (breakpoints) — may have attached themselves to code inside this module. The notifier chain gives them a chance to detach themselves before the module's memory is freed.

```
/* kernel/module.c — notifier chain */
/* Tell all registered listeners the module is about to disappear */
blocking_notifier_call_chain(&module_notify_list,
                             MODULE_STATE_GOING, mod);

/* Examples of who listens: ftrace, kprobes, perf, livepatch */
```

6.6 Step 5: Call Your exit Function

Now the kernel calls the exit function you wrote — the one registered with module_exit(). This is your chance to clean up everything you set up in init: free any memory you allocated, unregister any devices, remove any /proc entries. If you did not write an exit function, this step is skipped.

```
/* kernel/module.c and hello.c — the exit call */
/* kernel calls your hello_exit() */
if (mod->exit != NULL)
    mod->exit();

/* In your hello.c: */
static void __exit hello_exit(void)
{
    printk(KERN_INFO "[hello] Module removed.\n");
}
```

6.7 Step 6: Remove and Free Everything

free_module() is the final step. It unlinks the module from the global list, waits until all list readers are done (synchronize_rcu()), then frees every piece of memory associated with the module in order.

Where to find it: `kernel/module.c` — function `free_module()`

```
/* kernel/module.c — free_module() key steps */
list_del_rcu(&mod->list);     /* unlink from the modules list          */
synchronize_rcu();            /* wait: ensure no CPU is still reading  */
mod_sysfs_teardown(mod);      /* remove /sys/module/<name>/ directory  */
module_free(mod, mod->module_core); /* free the code from memory       */
kfree(mod);                   /* free the struct module itself         */
```

ℹ WHY SYNCHRONIZE_RCU() BEFORE FREEING?

list_del_rcu() removes the module from the list, but at that exact instant, another CPU might be in the middle of reading the list (for example, a lsmod call). synchronize_rcu() pauses until every such reader has finished. Only then is it safe to free the memory. Without this pause, a reader could follow a pointer to freed memory — which would crash the kernel.

6.8 The Pointer Surgery

Removing a node from a doubly-linked list is elegant — only two pointer updates are needed, on the two neighbouring nodes.

```
/* list_del_rcu() — the two pointer updates that unlink a node */
/* Before rmmod hello: */
/*  [ext4] <--> [hello] <--> [nfs]  */

/* list_del_rcu() updates exactly two pointers: */
hello->list.prev->next = hello->list.next; /* ext4 now points to nfs */
hello->list.next->prev = hello->list.prev; /* nfs  now points to ext4*/

/* After: */
/*  [ext4] <--> [nfs]               */
/*  hello is gone from the list,     */
/*  then freed from memory.          */
```

CHAPTER 7

Custom Modules We Built

Having studied the theory, we built two practical modules to prove it. The first replicates lsmod from inside kernel space. The second goes deeper: instead of writing a simulation, we modified the actual kernel source file (kernel/module.c) to emit a live trace during every real insmod on the system.

7.1 Module 1: lsmod Replicator

This module, when loaded with insmod, walks the same global modules linked list that lsmod reads — and prints a formatted table to dmesg. It shows every loaded module's name and size.

The Design

THIS_MODULE is a macro the compiler provides. It expands to a pointer to our own struct module. Since our freshly-loaded module is the last entry in the list, walking list.prev from THIS_MODULE lets us visit all previously loaded modules. We stop when we see THIS_MODULE again, which means we have completed the circle.

```
/* lsmod_rep.c — complete working module */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/list.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Sam");
MODULE_DESCRIPTION("Kernel-space lsmod replicator");

static int __init lsmod_rep_init(void)
{
    struct module *mod;
    int count = 0;

    printk(KERN_INFO "\n");
    printk(KERN_INFO "=== Loaded Kernel Modules ===\n");
    printk(KERN_INFO "%-30s %10s\n", "Module", "Size (bytes)");
    printk(KERN_INFO "%-30s %10s\n",
           "------------------------------", "----------");

    /* Walk backwards from THIS_MODULE through the circular list */
    list_for_each_entry(mod, THIS_MODULE->list.prev, list) {
        if (mod == THIS_MODULE) break;  /* full circle, stop here */
        printk(KERN_INFO "%-30s %10u\n", mod->name, mod->core_size);
        count++;
    }

    printk(KERN_INFO "--- Total: %d modules ---\n", count);
    return 0;
}

static void __exit lsmod_rep_exit(void)
{
    printk(KERN_INFO "lsmod_rep: unloaded.\n");
}
```

```
module_init(lsmod_rep_init);
module_exit(lsmod_rep_exit);
```

ℹ WHY CORE_SIZE AND NOT CORE_LAYOUT.SIZE?

This module was written for kernel 3.9. In that version, the field is called core_size directly on struct module. In newer kernels (4.5+), it moved to mod->core_layout.size. See Chapter 8 for the full compatibility story.

7.2 Module 2: Modifying kernel/module.c Directly

The lsmod replicator only runs when you load it. But what if you want to see the insmod stages automatically, every time any module is loaded on the system? For that, we went to the source: we opened kernel/module.c and added printk statements directly inside load_module() and do_init_module().

ℹ WHY THIS IS MORE POWERFUL THAN A STANDALONE MODULE

A module that prints "Stage 1 PASS" when you load it is a simulation. Modifying kernel/module.c means the printk calls fire inside the real kernel loading path — for every insmod, for every module, automatically. This is how kernel developers actually trace and debug the load process.

What Is kernel/module.c?

kernel/module.c is the single file in the Linux kernel source tree that contains all the module loading and unloading logic — sys_init_module(), load_module(), do_init_module(), sys_delete_module(), and free_module(). When you recompile the kernel after editing this file, your changes become a permanent part of the running kernel.

```
  Linux kernel source tree

  kernel/
    module.c   <-- we edited this file
      |
      +-- sys_init_module()    entry point for insmod
      +-- load_module()        all 10 verification stages live here
      +-- do_init_module()     calls your __init function
      +-- sys_delete_module()  entry point for rmmod
      +-- free_module()        frees all memory on removal
```

Figure: kernel/module.c — the file we modified

What We Added to load_module()

After each verification call inside load_module(), we added a single printk line. Here is the annotated version of the function showing where we placed each statement:

```
/* kernel/module.c — our printk additions inside load_module() */
/* kernel/module.c — load_module() with our printk additions */

err = copy_module_from_user(...);
if (err) return err;
printk(KERN_INFO "[INSMOD] Stage 1: Copy from userspace     - OK\n");

err = elf_header_check(info);
if (err) goto free_copy;
printk(KERN_INFO "[INSMOD] Stage 2: ELF header validation   - OK\n");

err = setup_load_info(info);
```

```
if (err) goto free_copy;
printk(KERN_INFO "[INSMOD] Stage 3: Find module sections     - OK\n");

err = check_modinfo(mod, info);
if (err) goto free_copy;
printk(KERN_INFO "[INSMOD] Stage 4: Version magic check      - OK\n");

err = layout_and_allocate(info, flags);
if (err) goto free_copy;
printk(KERN_INFO "[INSMOD] Stage 5: Allocate kernel memory   - OK\n");

err = apply_relocations(mod, info);
if (err) goto free_module;
printk(KERN_INFO "[INSMOD] Stage 6: Apply relocations        - OK\n");

err = simplify_symbols(mod, info);
if (err) goto free_module;
printk(KERN_INFO "[INSMOD] Stage 7: Resolve symbols          - OK\n");

err = security_module_check(mod);
if (err) goto free_module;
printk(KERN_INFO "[INSMOD] Stage 8: Security checks          - OK\n");

list_add_rcu(&mod->list, &modules);
printk(KERN_INFO "[INSMOD] Stage 9: Added to modules list    - OK\n");
```

What We Added to do_init_module()

Stage 10 happens in a separate function. We added a printk before and after the call to your init function so we could see your own printk firing in between them.

```
/* kernel/module.c — do_init_module() with our additions */
/* kernel/module.c — do_init_module() with our additions */

printk(KERN_INFO "[INSMOD] Stage 10: Calling module_init()...\n");

ret = do_one_initcall(mod->init);  /* YOUR hello_init() runs here */

if (ret < 0) {
    printk(KERN_ERR "[INSMOD] Stage 10: FAILED (err=%d)\n", ret);
    goto fail_free_freeinit;
}

printk(KERN_INFO "[INSMOD] Stage 10: module_init() returned  - OK\n");
printk(KERN_INFO "[INSMOD] Module '%s' is LIVE.\n", mod->name);
```

How to Rebuild the Kernel With Your Changes

```
# Shell commands to rebuild the kernel after editing module.c
# 1. Go to the kernel source directory
cd /usr/src/linux-3.9.0

# 2. Edit the file
vim kernel/module.c

# 3. Rebuild the kernel (this takes several minutes)
make -j$(nproc)
```

```
# 4. Install the new kernel
sudo make modules_install && sudo make install

# 5. Reboot into the new kernel
sudo reboot
```

What dmesg Shows After Every insmod

Once you boot into the modified kernel, every insmod — for any module — automatically produces this trace. Notice line [512.111]: your own printk from hello_init() fires between Stage 10 start and Stage 10 OK. That is the real execution order made visible.

```
# dmesg — your hello_init() printk fires inside Stage 10
$ sudo insmod hello.ko && dmesg | tail -15

[512.101] [INSMOD] Stage 1:  Copy from userspace     - OK
[512.102] [INSMOD] Stage 2:  ELF header validation   - OK
[512.103] [INSMOD] Stage 3:  Find module sections     - OK
[512.104] [INSMOD] Stage 4:  Version magic check      - OK
[512.105] [INSMOD] Stage 5:  Allocate kernel memory   - OK
[512.106] [INSMOD] Stage 6:  Apply relocations        - OK
[512.107] [INSMOD] Stage 7:  Resolve symbols          - OK
[512.108] [INSMOD] Stage 8:  Security checks          - OK
[512.109] [INSMOD] Stage 9:  Added to modules list    - OK
[512.110] [INSMOD] Stage 10: Calling module_init()...
[512.111] [hello]  Module loaded.      <-- your printk from hello_init()
[512.112] [INSMOD] Stage 10: module_init() returned   - OK
[512.113] [INSMOD] Module 'hello' is LIVE.
```

✦ **THIS IS HOW REAL KERNEL DEBUGGING WORKS**

Adding printk at key points in kernel source code is a fundamental debugging technique. ftrace and kprobes are the production-grade versions of the same idea — they let you attach observers to any function without modifying source code and recompiling. Once you understand what we did here, those tools will make much more sense.

CHAPTER 8

Kernel 3.9 Compatibility Notes

All the code in this document was written and tested on kernel 3.9.0. Kernel 3.9 is older than current releases, and some APIs changed in the years since. Here are the issues we ran into and how we fixed them. Anyone who tries to run this code on a different kernel version will likely hit the same problems.

8.1 core_size vs core_layout.size

This was the biggest compatibility issue. In kernel 3.9, the size of a module's core memory is a plain field directly on struct module. In kernel 4.5+, it was moved into a nested struct called module_layout.

Kernel version	How to access module size
3.x (this project)	mod->core_size
4.5 and later	mod->core_layout.size

```
/* Handling core_size vs core_layout.size */
/* Wrong on kernel 3.9 — core_layout does not exist yet: */
printk("%u", mod->core_layout.size);   /* compile error */

/* Correct for kernel 3.9: */
printk("%u", mod->core_size);

/* To write code that works on both: */
#if LINUX_VERSION_CODE >= KERNEL_VERSION(4, 5, 0)
    printk("%u", mod->core_layout.size);
#else
    printk("%u", mod->core_size);
#endif
```

8.2 printk Format Specifier Strictness

The kernel build system treats format specifier mismatches as errors. Using %lu for an unsigned int (or %d for a size_t) will break the build. The rule is simple: match the specifier exactly to the type.

```
/* Format specifier correctness */
/* Wrong — %lu is for unsigned long, core_size is unsigned int: */
printk("%lu", mod->core_size);    /* warning, treated as error */

/* Correct: */
printk("%u", mod->core_size);     /* %u for unsigned int */
```

8.3 List Traversal Direction

Our first version of the lsmod replicator walked list.next instead of list.prev from THIS_MODULE. Because our module was the last one loaded, walking .next went straight to the list anchor — skipping almost all modules. The fix is to walk .prev.

```
/* List traversal direction fix */
/* Bug: walking .next from THIS_MODULE wraps to anchor immediately */
```

```
list_for_each_entry(mod, THIS_MODULE->list.next, list) { ... }

/* Fix: walk .prev to visit all previously loaded modules */
list_for_each_entry(mod, THIS_MODULE->list.prev, list) {
    if (mod == THIS_MODULE) break;  /* completed the circle */
}
```

8.4 Unicode Characters in Kernel Output

We tried using Unicode box-drawing characters (like ╔, ═, ╗) inside printk strings to make nicer output in dmesg. They showed up as garbled bytes or question marks. The kernel console does not reliably support multi-byte Unicode. Use plain ASCII only.

What we tried	Problem	Solution
Unicode box chars ╔═╗	Garbled output in dmesg	Use ASCII: +--+ | -
Check mark ✓	Outside printable ASCII	Use: OK or PASS
Arrow →	Encoding issue	Use: ->

CHAPTER 9

Lifecycle Summary and Quick Reference

This chapter brings together the complete picture of a kernel module's life — from the moment you write the source file to the moment the kernel frees its last byte of memory.

```
LINUX KERNEL MODULE — COMPLETE LIFECYCLE
=========================================

[1]  You write hello.c  +  Makefile
      |
      |  $ make
      v
[2]  kbuild compiles -> hello.ko  (an ELF relocatable object)
      |
      |  $ sudo insmod hello.ko
      v
[3]  insmod opens file, maps it, calls init_module() syscall
      |
      v  (enters kernel space — kernel/module.c takes over)
[4]  load_module() runs all 10 stages:
      Stage 1: copy_module_from_user()  -- copy .ko into kernel mem
      Stage 2: elf_header_check()       -- validate ELF magic bytes
      Stage 3: setup_load_info()        -- find .this_module section
      Stage 4: check_modinfo()          -- vermagic check
      Stage 5: layout_and_allocate()    -- allocate core + init memory
      Stage 6: apply_relocations()      -- patch placeholder addresses
      Stage 7: simplify_symbols()       -- look up printk, kmalloc...
      Stage 8: security_module_check()  -- LSM / signature check
      Stage 9: list_add_rcu()           -- add to global modules list
      |
      v
[5]  do_init_module() calls hello_init()
      mod->state = MODULE_STATE_LIVE
      free_module_init() frees the .init.text section
      |
      |  (module is now live — lsmod shows it)
      |
      |  $ sudo rmmod hello
      v
[6]  sys_delete_module():
      find_module("hello")           -- locate in linked list
      module_refcount(mod) == 0 ?    -- ensure nothing depends on it
      mod->state = MODULE_STATE_GOING
      blocking_notifier_call_chain() -- warn ftrace, kprobes, etc.
      mod->exit()                    -- your hello_exit() runs
      |
      v
[7]  free_module():
      list_del_rcu(&mod->list)       -- unlink from the list
      synchronize_rcu()              -- wait for all list readers
      module_free(module_core)       -- free code + data
      kfree(mod)                     -- free struct module
      Module is completely gone.
```

Figure: Complete kernel module lifecycle — writing through removal

9.1 Key Commands

Command	What it does
make	Compile your .c file into a .ko kernel module using kbuild
sudo insmod hello.ko	Load the module into the running kernel
lsmod	List all currently loaded modules (reads /proc/modules)
dmesg | tail -20	View the last 20 kernel log messages — your printk output
sudo rmmod hello	Remove the module from the kernel
modinfo hello.ko	Show the module's metadata: license, author, vermagic
cat /proc/modules	Raw content that lsmod formats into its table
readelf -a hello.ko	Inspect ELF sections, symbols, and relocation entries

9.2 Key Concepts

Term	What it means
Kernel Module (.ko)	A compiled file that the running kernel can load and unload without rebooting
struct module	The kernel's internal record for a loaded module — holds its name, size, init/exit pointers, and list node
list_head	A two-pointer struct (next, prev) embedded inside data structures to form linked lists
modules (global)	The global anchor for the circular doubly-linked list of all loaded modules
container_of()	Calculates a struct module pointer from a pointer to its embedded list_head
vermagic	A version string in every .ko that must match the running kernel exactly, or insmod rejects the module
RCU	Read-Copy-Update: a technique allowing concurrent lock-free readers of the modules list
__init / __exit	__init places code in a section freed after loading; __exit places code that runs on rmmod
MODULE_LICENSE()	Declares the licence — GPL is needed to access all kernel-exported functions
printk / dmesg	printk() writes to the kernel ring buffer; dmesg shows that buffer from the terminal
load_module()	The central function in kernel/module.c that runs all 10 insmod stages
do_init_module()	Called last by load_module(); runs your __init function and sets state to LIVE
free_module()	Called by rmmod path; unlinks from list and frees all memory in the correct order